# تمرین جدید پایتون همراه با پاسخ کامل

۴ شهریور ۱۴۰۵

## فهرست مطالب


**۱ راهنمای استفاده از مجموعه ۱**

**۲ بخش اول: شرط‌ها و منطق کاربردی ۱**

۱.۲ تمرین ۱: ارزیابی قدرت رمز عبور ۱

۲.۲ تمرین ۲: تخفیف پلکانی خرید ۱

۳.۲ تمرین ۳: اعتبارسنجی ساعت ۱

۴.۲ تمرین ۴: هزینه ارسال ۱

۵.۲ تمرین ۵: تعیین شیفت کاری ۲

۶.۲ تمرین ۶: هزینه پارکینگ ۲

۷.۲ تمرین ۷: وضعیت موجودی محصول ۲

۸.۲ تمرین ۸: تشخیص سال کبیسه ۲

۹.۲ تمرین ۹: محاسبه اضافه‌کاری ۲

۱۰.۲ تمرین ۱۰: تعیین بازه دمایی ۲

**۳ بخش دوم: حلقه‌ها و شبیه‌سازی ۳**

۱.۳ تمرین ۱۱: پس‌انداز ماهانه ۳

۲.۳ تمرین ۱۲: شمارش معکوس پویا ۳

۳.۳ تمرین ۱۳: شبیه‌سازی رشد جمعیت ۳

۴.۳ تمرین ۱۴: دستگاه فروش بلیت ۳

۵.۳ تمرین ۱۵: محاسبه بهره مرکب ۴

۶.۳ تمرین ۱۶: تولید الگوی پله‌ای شماره‌دار ۴

۷.۳ تمرین ۱۷: کنترل تلاش ورود ۴

۸.۳ تمرین ۱۸: شبیه‌سازی مصرف باتری ۴

۹.۳ تمرین ۱۹: یافتن نخستین مضرب مشترک پس از حد ۴

۱۰.۳ تمرین ۲۰: فروش روزانه تا رسیدن به هدف ۴

**۴ بخش سوم: پردازش متن و رشته‌ها ۵**

۱.۴ تمرین ۲۱: پاک‌سازی فاصله‌های اضافی ۵

۲.۴ تمرین ۲۲: شمارش هشتگ‌های معتبر ۵

۳.۴ تمرین ۲۳: پنهان‌سازی شماره کارت ۵

۴.۴ تمرین ۲۴: ساخت نام کاربری ۵

۵.۴ تمرین ۲۵: استخراج دامنه ایمیل ۵

۶.۴ تمرین ۲۶: کوتاه‌سازی عنوان ۶

۷.۴ تمرین ۲۷: شمارش جمله‌ها ۶

۸.۴ تمرین ۲۸: ساخت سرواژه ۶

۹.۴ تمرین ۲۹: تشخیص حروف تکراری غیرمجاز ۶

۱۰.۴ تمرین ۳۰: قالب‌بندی عنوان کتاب ۶

**۵ بخش چهارم: فهرست، دیکشنری و مجموعه ۶**

۱.۵ تمرین ۳۱: ادغام موجودی انبار ۶

۲.۵ تمرین ۳۲: گروه‌بندی نام‌ها بر اساس حرف اول ۷

۳.۵ تمرین ۳۳: یافتن کالاهای کم‌موجود ۷

۴.۵ تمرین ۳۴: اشتراک مهارت‌های اعضای تیم ۷

۵.۵ تمرین ۳۵: رتبه‌بندی فروشندگان ۷

۶.۵ تمرین ۳۶: تقسیم فهرست به بسته‌های ثابت ۷

۷.۵ تمرین ۳۷: چرخش فهرست ۷

۸.۵ تمرین ۳۸: ساخت فهرست انتظار بدون تکرار . . . . . ۸
۹.۵ تمرین ۳۹: ماتریس فروش هفتگی . . . . . ۸
۱۰.۵ تمرین ۴۰: اختلاف دو سبد خرید . . . . . ۸

**۶ بخش پنجم: توابع و الگوریتم‌ها ۸**
۱.۶ تمرین ۴۱: محاسبه قیمت پس از مالیات . . . . . ۸
۲.۶ تمرین ۴۲: تبدیل بازه امتیاز . . . . . ۸
۳.۶ تمرین ۴۳: جست‌وجوی نزدیک‌ترین مقدار . . . . . ۹
۴.۶ تمرین ۴۴: فشرده‌سازی بازه‌های عددی . . . . . ۹
۵.۶ تمرین ۴۵: محاسبه مبلغ اقساط مساوی . . . . . ۹
۶.۶ تمرین ۴۶: ادغام بازه‌های هم‌پوشان . . . . . ۹
۷.۶ تمرین ۴۷: بررسی تعادل پرانتزها . . . . . ۹
۸.۶ تمرین ۴۸: محاسبه زمان انتظار میانگین . . . . . ۱۰
۹.۶ تمرین ۴۹: یافتن طولانی‌ترین روند افزایشی پیوسته . . . . . ۱۰
۱۰.۶ تمرین ۵۰: تخصیص بودجه تناسبی . . . . . ۱۰

**۷ بخش ششم: اعتبارسنجی و مدیریت خطا ۱۰**
۱.۷ تمرین ۵۱: دریافت عدد صحیح در بازه . . . . . ۱۱
۲.۷ تمرین ۵۲: اعتبارسنجی شماره تلفن . . . . . ۱۱
۳.۷ تمرین ۵۳: اعتبارسنجی تاریخ میلادی . . . . . ۱۱
۴.۷ تمرین ۵۴: تقسیم امن داده‌های متنی . . . . . ۱۱
۵.۷ تمرین ۵۵: اعتبارسنجی کد محصول . . . . . ۱۱
۶.۷ تمرین ۵۶: دریافت انتخاب از منو . . . . . ۱۲
۷.۷ تمرین ۵۷: اعتبارسنجی نشانی IPv4 . . . . . ۱۲
۸.۷ تمرین ۵۸: تبدیل مبلغ با پیام خطای دقیق . . . . . ۱۲
۹.۷ تمرین ۵۹: بررسی مسیر فایل ورودی . . . . . ۱۲
۱۰.۷ تمرین ۶۰: اعتبارسنجی رکورد کاربر . . . . . ۱۲

**۸ بخش هفتم: فایل، CSV و JSON ۱۳**
۱.۸ تمرین ۶۱: شمارش خطوط غیرخالی فایل . . . . . ۱۳
۲.۸ تمرین ۶۲: افزودن گزارش رویداد . . . . . ۱۳
۳.۸ تمرین ۶۳: محاسبه مجموع فروش از CSV . . . . . ۱۳
۴.۸ تمرین ۶۴: فیلتر CSV و ساخت فایل جدید . . . . . ۱۳
۵.۸ تمرین ۶۵: به‌روزرسانی تنظیمات JSON . . . . . ۱۳
۶.۸ تمرین ۶۶: پشتیبان‌گیری از فایل . . . . . ۱۴
۷.۸ تمرین ۶۷: یافتن بزرگ‌ترین فایل پوشه . . . . . ۱۴
۸.۸ تمرین ۶۸: تبدیل CSV به JSON . . . . . ۱۴
۹.۸ تمرین ۶۹: استخراج کلمات خطا از گزارش . . . . . ۱۴
۱۰.۸ تمرین ۷۰: گزارش فراوانی پسوند فایل‌ها . . . . . ۱۵

**۹ بخش هشتم: شیءگرایی کاربردی ۱۵**
۱.۹ تمرین ۷۱: کیف پول دیجیتال . . . . . ۱۵
۲.۹ تمرین ۷۲: محصول با تخفیف . . . . . ۱۵
۳.۹ تمرین ۷۳: حساب بانکی با تاریخچه . . . . . ۱۵
۴.۹ تمرین ۷۴: کتاب امانتی . . . . . ۱۶
۵.۹ تمرین ۷۵: سبد خرید . . . . . ۱۶
۶.۹ تمرین ۷۶: کارمند و حقوق خالص . . . . . ۱۷
۷.۹ تمرین ۷۷: رزرو صندلی . . . . . ۱۷
۸.۹ تمرین ۷۸: دماسنج با ویژگی محاسباتی . . . . . ۱۷
۹.۹ تمرین ۷۹: اعلان‌های چندنوعی . . . . . ۱۷
۱۰.۹ تمرین ۸۰: دانشجو با میانگین وزنی . . . . . ۱۸

**۱۰ بخش نهم: ساختمان داده و حل مسئله ۱۸**
۱.۱۰ تمرین ۸۱: صف پشتیبانی . . . ۱۸
۲.۱۰ تمرین ۸۲: تاریخچه مرورگر با پشته . . . ۱۸
۳.۱۰ تمرین ۸۳: جست‌وجوی دودویی . . . ۱۸
۴.۱۰ تمرین ۸۴: مرتب‌سازی انتخابی . . . ۱۹
۵.۱۰ تمرین ۸۵: پیمایش شبکه با BFS . . . ۱۹
۶.۱۰ تمرین ۸۶: شمارش فراوانی با Counter . . . ۱۹
۷.۱۰ تمرین ۸۷: صف اولویت کارها . . . ۱۹
۸.۱۰ تمرین ۸۸: حافظه نهان ساده . . . ۲۰
۹.۱۰ تمرین ۸۹: یافتن مسیر در گراف . . . ۲۰
۱۰.۱۰ تمرین ۹۰: بیشترین درآمد بدون انتخاب روزهای مجاور . . . ۲۰

**۱۱ بخش دهم: مینی‌پروژه‌های کاربردی ۲۰**
۱.۱۱ تمرین ۹۱: مدیریت هزینه‌های روزانه . . . ۲۱
۲.۱۱ تمرین ۹۲: فهرست کارها . . . ۲۱
۳.۱۱ تمرین ۹۳: نظرسنجی ساده . . . ۲۱
۴.۱۱ تمرین ۹۴: دفترچه مخاطبان . . . ۲۱
۵.۱۱ تمرین ۹۵: سیستم ورود با قفل موقت . . . ۲۲
۶.۱۱ تمرین ۹۶: صدور فاکتور فروش . . . ۲۲
۷.۱۱ تمرین ۹۷: تحلیل نمرات کلاس . . . ۲۲
۸.۱۱ تمرین ۹۸: رزرو اتاق هتل . . . ۲۳
۹.۱۱ تمرین ۹۹: تحلیل کپشن . . . ۲۳
۱۰.۱۱ تمرین ۱۰۰: مدیریت سفارش رستوران . . . ۲۳

# ۱ راهنمای استفاده از مجموعه

این مجموعه ۱۰۰ مسئله تازه و کاربردی را ارائه می‌کند تا مهارت حل مسئله با پایتون را از تصمیم‌گیری‌های ساده تا پروژه‌های چندمرحله‌ای تقویت کند. تمرین‌ها با توجه به موضوع‌ها و الگوهای پرتکرار دو مجموعه مرجع طراحی شده‌اند و از بازنویسی مستقیم سؤال‌های آن‌ها پرهیز شده است.

**تهیه و تنظیم: مهشید شیدایی**
**نسخه برندشده برای صفحه «مهشید شیدایی»**

# ۲ بخش اول: شرط‌ها و منطق کاربردی

تمرین‌های این بخش تصمیم‌گیری‌های روزمره را با شرط‌های روشن مدل می‌کنند.

## ۱.۲ تمرین ۱: ارزیابی قدرت رمز عبور

رمزی را دریافت کنید. اگر حداقل ۸ نویسه و دست‌کم یک حرف بزرگ، یک حرف کوچک و یک رقم داشت، آن را قوی اعلام کنید.

```
password = input("Enter password: ")
strong = (len(password) >= 8 and any(c.isupper() for c in password)
          and any(c.islower() for c in password)
          and any(c.isdigit() for c in password))
print("Strong password" if strong else "Weak password")
```

## ۲.۲ تمرین ۲: تخفیف پلکانی خرید

مبلغ خرید را بگیرید؛ برای مبلغ‌های حداقل ۵۰۰، ٪۲۰ و برای حداقل ۲۰۰، ٪۱۰ تخفیف اعمال کنید.

```
amount = float(input("Enter purchase amount: "))
rate = 0.20 if amount >= 500 else 0.10 if amount >= 200 else 0
print(f"Final amount: {amount * (1 - rate):.2f}")
```

### ۳.۲ تمرین ۳: اعتبارسنجی ساعت

ساعت و دقیقه را بگیرید و معتبر بودن زمان را بررسی کنید.

```
hour = int(input("Enter hour: "))
minute = int(input("Enter minute: "))
print("Valid time" if 0 <= hour <= 23 and 0 <= minute <= 59 else "Invalid time")
```

### ۴.۲ تمرین ۴: هزینه ارسال

اگر مبلغ سفارش حداقل ۱۰۰ باشد ارسال رایگان است؛ در غیر این صورت، برای سفارش فوری ۱۵ و سفارش عادی ۷ واحد هزینه بگیرید.

```
amount = float(input("Enter order amount: "))
express = input("Express delivery? (yes/no): ").lower() == "yes"
fee = 0 if amount >= 100 else 15 if express else 7
print(f"Shipping fee: {fee}")
```

### ۵.۲ تمرین ۵: تعیین شیفت کاری

ساعت شروع را دریافت و شیفت صبح، عصر یا شب را مشخص کنید.

```
hour = int(input("Enter start hour: "))
if 6 <= hour < 14:
    shift = "Morning"
elif 14 <= hour < 22:
    shift = "Evening"
else:
    shift = "Night"
print(f"Shift: {shift}")
```

### ۶.۲ تمرین ۶: هزینه پارکینگ

برای ساعت اول ۵، ساعت‌های دوم تا پنجم هرکدام ۳ و ساعت‌های بعدی هرکدام ۲ واحد محاسبه کنید.

```
hours = int(input("Enter parked hours: "))
if hours <= 0:
    fee = 0
elif hours == 1:
    fee = 5
elif hours <= 5:
    fee = 5 + (hours - 1) * 3
else:
    fee = 17 + (hours - 5) * 2
print(f"Parking fee: {fee}")
```

### ۷.۲ تمرین ۷: وضعیت موجودی محصول

موجودی و نقطه سفارش را بگیرید و وضعیت را «ناموجود»، «نیازمند سفارش» یا «کافی» اعلام کنید.

```
stock = int(input("Enter stock: "))
reorder_level = int(input("Enter reorder level: "))
status = "Out of stock" if stock == 0 else "Reorder needed" if stock <= reorder_level else "Sufficient stock"
print(status)
```

### ۸.۲ تمرین ۸: تشخیص سال کبیسه

سال میلادی را طبق قواعد کامل سال کبیسه بررسی کنید.

```
year = int(input("Enter year: "))
leap = year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
print("Leap year" if leap else "Common year")
```

### ۹.۲ تمرین ۹: محاسبه اضافه‌کاری

ساعات کار و نرخ ساعتی را بگیرید؛ ساعات بیشتر از ۴۰ را با ضریب ۱٫۵ محاسبه کنید.

```
hours = float(input("Enter worked hours: "))
rate = float(input("Enter hourly rate: "))
pay = hours * rate if hours <= 40 else 40 * rate + (hours - 40) * rate * 1.5
print(f"Weekly pay: {pay:.2f}")
```

### ۱۰.۲ تمرین ۱۰: تعیین بازه دمایی

دما را به یکی از وضعیت‌های یخبندان، سرد، معتدل یا گرم دسته‌بندی کنید.

```
temperature = float(input("Enter temperature: "))
if temperature < 0:
    label = "Freezing"
elif temperature < 15:
    label = "Cold"
elif temperature < 30:
    label = "Mild"
else:
    label = "Hot"
print(label)
```

## ۳ بخش دوم: حلقه‌ها و شبیه‌سازی

تمرین‌های این بخش تکرار، رشد مرحله‌ای و توقف بر اساس شرط را در سناریوهای کاربردی تمرین می‌کنند.
تمرین‌های این بخش تصمیم‌گیری‌های روزمره را با شرط‌های روشن مدل می‌کنند.

### ۱.۳ تمرین ۱۱: پس‌انداز ماهانه

سرمایه اولیه، واریز ماهانه و تعداد ماه‌ها را بگیرید و موجودی هر ماه را نمایش دهید.

```
balance = float(input("Enter initial balance: "))
deposit = float(input("Enter monthly deposit: "))
months = int(input("Enter number of months: "))
for month in range(1, months + 1):
    balance += deposit
    print(f"Month {month}: {balance:.2f}")
```

### ۲.۳ تمرین ۱۲: شمارش معکوس پویا

از عدد ورودی تا صفر با گام ۱ شمارش معکوس کنید.

```
start = int(input("Enter start number: "))
for value in range(start, -1, -1):
    print(value)
print("Done")
```

### ۳.۳ تمرین ۱۳: شبیه‌سازی رشد جمعیت

جمعیت اولیه و نرخ رشد را بگیرید و نخستین سال عبور از هدف را پیدا کنید.

```
population = int(input("Enter initial population: "))
rate = float(input("Enter annual growth percent: "))
target = int(input("Enter target population: "))
years = 0
while population < target:
```

```
    population = int(population * (1 + rate / 100))
    years += 1
print(f"Years needed: {years}")
```

### ۴.۳ تمرین ۱۴: دستگاه فروش بلیت

تا زمانی که ظرفیت باقی مانده است، تعداد بلیت درخواستی مشتریان را دریافت و فروش را ثبت کنید.

```
remaining = int(input("Enter ticket capacity: "))
while remaining > 0:
    requested = int(input("Enter requested tickets: "))
    sold = min(max(requested, 0), remaining)
    remaining -= sold
    print(f"Sold: {sold}, Remaining: {remaining}")
```

### ۵.۳ تمرین ۱۵: محاسبه بهره مرکب

موجودی پایان هر سال را با نرخ ثابت چاپ کنید.

```
balance = float(input("Enter principal: "))
rate = float(input("Enter annual rate percent: "))
years = int(input("Enter years: "))
for year in range(1, years + 1):
    balance *= 1 + rate / 100
    print(f"Year {year}: {balance:.2f}")
```

### ۶.۳ تمرین ۱۶: تولید الگوی پله‌ای شماره‌دار

برای عدد `n`، در سطر `i` اعداد ۱ تا `i` را چاپ کنید.

```
rows = int(input("Enter number of rows: "))
for row in range(1, rows + 1):
    print(" ".join(str(value) for value in range(1, row + 1)))
```

### ۷.۳ تمرین ۱۷: کنترل تلاش ورود

رمز صحیح را حداکثر در سه تلاش دریافت کنید.

```
correct_password = "python123"
for attempt in range(1, 4):
    password = input("Enter password: ")
    if password == correct_password:
        print("Access granted")
        break
    print(f"Attempts left: {3 - attempt}")
else:
    print("Account locked")
```

### ۸.۳ تمرین ۱۸: شبیه‌سازی مصرف باتری

در هر چرخه درصدی از باتری کم کنید تا به آستانه هشدار برسد.

```
battery = float(input("Enter battery percent: "))
drain = float(input("Enter drain per cycle: "))
cycles = 0
while battery > 20:
    battery = max(0, battery - drain)
    cycles += 1
print(f"Warning after {cycles} cycles at {battery:.1f}%")
```

### ۹.۳ تمرین ۱۹: یافتن نخستین مضرب مشترک پس از حد

دو عدد و یک حد را بگیرید و نخستین عدد پس از حد را که بر هر دو بخش‌پذیر است پیدا کنید.

```
a = int(input("Enter first divisor: "))
b = int(input("Enter second divisor: "))
value = int(input("Enter lower bound: ")) + 1
while value % a != 0 or value % b != 0:
    value += 1
print(f"First matching value: {value}")
```

### ۱۰.۳ تمرین ۲۰: فروش روزانه تا رسیدن به هدف

فروش روزها را دریافت کنید و روز رسیدن مجموع فروش به هدف را اعلام کنید.

```
target = float(input("Enter sales target: "))
total = 0
day = 0
while total < target:
    day += 1
    total += float(input(f"Enter sales for day {day}: "))
print(f"Target reached on day {day}")
```

## ۴ بخش سوم: پردازش متن و رشته‌ها

تمرین‌های این بخش بر پاک‌سازی، استخراج و قالب‌بندی داده‌های متنی تمرکز دارند.

### ۱.۴ تمرین ۲۱: پاک‌سازی فاصله‌های اضافی

فاصله‌های ابتدا، انتها و میان کلمات را استاندارد کنید.

```
text = input("Enter text: ")
cleaned = " ".join(text.split())
print(cleaned)
```

### ۲.۴ تمرین ۲۲: شمارش هشتگ‌های معتبر

هشتگ‌هایی را بشمارید که با # شروع می‌شوند و پس از آن دست‌کم یک نویسه دارند.

```
caption = input("Enter caption: ")
hashtags = [word for word in caption.split() if word.startswith("#") and len(word) > 1]
print(f"Hashtag count: {len(hashtags)}")
```

### ۳.۴ تمرین ۲۳: پنهان‌سازی شماره کارت

همه رقم‌ها به‌جز چهار رقم آخر را با * جایگزین کنید.

```
card = input("Enter card number: ").replace(" ", "")
masked = "*" * max(0, len(card) - 4) + card[-4:]
print(masked)
```

### ۴.۴ تمرین ۲۴: ساخت نام کاربری

نام و نام خانوادگی را پاک‌سازی و نام کاربری `firstname.lastname` ایجاد کنید.

```
first_name = input("Enter first name: ").strip().lower()
last_name = input("Enter last name: ").strip().lower()
username = f"{first_name}.{last_name}".replace(" ", "")
print(username)
```

### ۵.۴ تمرین ۲۵: استخراج دامنه ایمیل

بخش پس از @ را از ایمیل معتبر استخراج کنید.

```
email = input("Enter email: ").strip()
if email.count("@") == 1 and not email.startswith("@") and not email.endswith("@"):
    print(f"Domain: {email.split('@')[1]}")
else:
    print("Invalid email")
```

### ۶.۴ تمرین ۲۶: کوتاه‌سازی عنوان

اگر عنوان از حد تعیین‌شده بلندتر بود، آن را قطع و سه نقطه اضافه کنید.

```
title = input("Enter title: ")
limit = int(input("Enter character limit: "))
shortened = title if len(title) <= limit else title[:max(0, limit - 3)] + "..."
print(shortened)
```

### ۷.۴ تمرین ۲۷: شمارش جمله‌ها

تعداد جمله‌هایی را که با ., ! یا ? پایان می‌یابند محاسبه کنید.

```
text = input("Enter text: ")
count = sum(1 for char in text if char in ".!?")
print(f"Sentence count: {count}")
```

### ۸.۴ تمرین ۲۸: ساخت سرواژه

از حرف اول کلمات یک عبارت، سرواژه بزرگ بسازید.

```
phrase = input("Enter phrase: ")
acronym = "".join(word[0].upper() for word in phrase.split() if word)
print(acronym)
```

### ۹.۴ تمرین ۲۹: تشخیص حروف تکراری غیرمجاز

مشخص کنید آیا یک نام کاربری دارای دو نویسه یکسانِ پشت‌سرهم است یا خیر.

```
username = input("Enter username: ")
has_repeat = any(username[i] == username[i + 1] for i in range(len(username) - 1))
print("Repeated pair found" if has_repeat else "No repeated pair")
```

### ۱۰.۴ تمرین ۳۰: قالب‌بندی عنوان کتاب

کلمات را با حرف اول بزرگ نمایش دهید، اما حروف ربط مشخص در میانه عنوان کوچک بمانند.

```
title = input("Enter book title: ")
small_words = {"and", "or", "of", "the", "in"}
words = title.lower().split()
result = []
for index, word in enumerate(words):
    result.append(word if index > 0 and word in small_words else word.capitalize())
print(" ".join(result))
```

## ۵ بخش چهارم: فهرست، دیکشنری و مجموعه

تمرین‌های این بخش سازمان‌دهی و تحلیل داده‌های چندتایی را با ساختارهای داخلی پایتون تمرین می‌کنند.

### ۱.۵ تمرین ۳۱: ادغام موجودی انبار

موجودی دو انبار را برای کالاهای هم‌نام جمع کنید.

```
warehouse_a = {"pen": 12, "book": 5, "bag": 2}
warehouse_b = {"pen": 8, "notebook": 6, "bag": 3}
merged = warehouse_a.copy()
for product, quantity in warehouse_b.items():
    merged[product] = merged.get(product, 0) + quantity
print(merged)
```

### ۲.۵ تمرین ۳۲: گروه‌بندی نام‌ها بر اساس حرف اول

نام‌ها را در دیکشنری بر اساس حرف اول دسته‌بندی کنید.

```
names = input("Enter names separated by commas: ").split(",")
groups = {}
for name in (item.strip() for item in names):
    if name:
        groups.setdefault(name[0].upper(), []).append(name)
print(groups)
```

### ۳.۵ تمرین ۳۳: یافتن کالاهای کم‌موجود

کالاهایی را نمایش دهید که موجودی آن‌ها کمتر از آستانه است.

```
inventory = {"keyboard": 4, "mouse": 12, "monitor": 2, "cable": 20}
threshold = int(input("Enter stock threshold: "))
low_stock = {name: stock for name, stock in inventory.items() if stock < threshold}
print(low_stock)
```

### ۴.۵ تمرین ۳۴: اشتراک مهارت‌های اعضای تیم

مهارت‌هایی را بیابید که همه اعضا دارند.

```
team_skills = [
    {"python", "git", "sql"},
    {"python", "docker", "git"},
    {"python", "git", "testing"}
]
common = set.intersection(*team_skills) if team_skills else set()
print(sorted(common))
```

### ۵.۵ تمرین ۳۵: رتبه‌بندی فروشندگان

نام و فروش را در یک دیکشنری قرار دهید و سه فروشنده برتر را نمایش دهید.

```
sales = {"Alice": 820, "Bob": 640, "Carol": 910, "David": 760}
ranking = sorted(sales.items(), key=lambda item: item[1], reverse=True)[:3]
for rank, (name, amount) in enumerate(ranking, 1):
    print(f"{rank}. {name}: {amount}")
```

### ۶.۵ تمرین ۳۶: تقسیم فهرست به بسته‌های ثابت

فهرست را به زیرفهرست‌هایی با اندازه مشخص تقسیم کنید.

```
values = input("Enter values separated by spaces: ").split()
size = int(input("Enter chunk size: "))
chunks = [values[i:i + size] for i in range(0, len(values), size)] if size > 0 else []
print(chunks)
```

### ۷.۵ تمرین ۳۷: چرخش فهرست

فهرست را به‌اندازه k خانه به راست بچرخانید.

```
values = input("Enter values separated by spaces: ").split()
k = int(input("Enter rotation count: "))
if values:
    k %= len(values)
    values = values[-k:] + values[:-k] if k else values
print(values)
```

### ۸.۵ تمرین ۳۸: ساخت فهرست انتظار بدون تکرار

نام‌های ورودی را با حفظ ترتیب نخستین حضور، در صف انتظار قرار دهید.

```
entries = input("Enter names separated by commas: ").split(",")
waiting_list = []
seen = set()
for entry in entries:
    name = entry.strip()
    if name and name.lower() not in seen:
        waiting_list.append(name)
        seen.add(name.lower())
print(waiting_list)
```

### ۹.۵ تمرین ۳۹: ماتریس فروش هفتگی

مجموع فروش هر ردیف را برای چند فروشگاه محاسبه کنید.

```
sales = [
    [120, 95, 110, 130, 150, 180, 160],
    [80, 90, 85, 100, 120, 140, 125]
]
for index, weekly_sales in enumerate(sales, 1):
    print(f"Store {index}: {sum(weekly_sales)}")
```

### ۱۰.۵ تمرین ۴۰: اختلاف دو سبد خرید

کالاهای مختص هر سبد را جداگانه نمایش دهید.

```
cart_a = set(input("Enter first cart items: ").split())
cart_b = set(input("Enter second cart items: ").split())
print(f"Only first cart: {sorted(cart_a - cart_b)}")
print(f"Only second cart: {sorted(cart_b - cart_a)}")
```

## ۶ بخش پنجم: توابع و الگوریتم‌ها

تمرین‌های این بخش بر طراحی تابع‌های مستقل و قابل‌آزمون تمرکز دارند.

### ۱.۶ تمرین ۴۱: محاسبه قیمت پس از مالیات

تابعی بنویسید که قیمت و درصد مالیات را دریافت و مبلغ نهایی را برگرداند.

```
def final_price(price, tax_percent):
    return round(price * (1 + tax_percent / 100), 2)

print(final_price(80, 9))
```

## ۲.۶ تمرین ۴۲: تبدیل بازه امتیاز

امتیازی از یک مقیاس را به مقیاس دیگری تبدیل کنید.

```
def convert_score(score, old_max, new_max):
    if old_max <= 0:
        raise ValueError("Old maximum must be positive")
    return round(score / old_max * new_max, 2)

print(convert_score(17, 20, 100))
```

## ۳.۶ تمرین ۴۳: جست‌وجوی نزدیک‌ترین مقدار

نزدیک‌ترین عدد فهرست به هدف را پیدا کنید.

```
def closest_value(numbers, target):
    if not numbers:
        return None
    return min(numbers, key=lambda number: (abs(number - target), number))

print(closest_value([4, 11, 18, 25], 15))
```

## ۴.۶ تمرین ۴۴: فشرده‌سازی بازه‌های عددی

اعداد مرتب را به بازه‌هایی مانند `1-3, 7-8` تبدیل کنید.

```
def compress_ranges(numbers):
    if not numbers:
        return ""
    numbers = sorted(set(numbers))
    ranges = []
    start = previous = numbers[0]
    for number in numbers[1:]:
        if number != previous + 1:
            ranges.append(str(start) if start == previous else f"{start}-{previous}")
            start = number
        previous = number
    ranges.append(str(start) if start == previous else f"{start}-{previous}")
    return ", ".join(ranges)

print(compress_ranges([1, 2, 3, 7, 8, 10]))
```

## ۵.۶ تمرین ۴۵: محاسبه مبلغ اقساط مساوی

مبلغ، پیش‌پرداخت و تعداد اقساط را دریافت و مبلغ هر قسط را محاسبه کنید.

```
def installment_amount(total, down_payment, count):
    if count <= 0 or down_payment > total:
        raise ValueError("Invalid installment data")
    return round((total - down_payment) / count, 2)

print(installment_amount(1200, 300, 6))
```

## ۶.۶ تمرین ۴۶: ادغام بازه‌های هم‌پوشان

بازه‌های زمانی هم‌پوشان را با یکدیگر ادغام کنید.

```
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
```

```
        if not merged or start > merged[-1][1]:
            merged.append([start, end])
        else:
            merged[-1][1] = max(merged[-1][1], end)
    return merged

print(merge_intervals([(1, 4), (3, 6), (8, 10)]))
```

## ۷.۶ تمرین ۴۷: بررسی تعادل پرانتزها

درستی ترتیب ()، [] و {} را بررسی کنید.

```
def balanced_brackets(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for char in text:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
    return not stack

print(balanced_brackets("{[()]}"))
```

## ۸.۶ تمرین ۴۸: محاسبه زمان انتظار میانگین

زمان ورود و شروع خدمت مشتریان را دریافت کنید.

```
def average_wait_time(arrivals, starts):
    if len(arrivals) != len(starts) or not arrivals:
        raise ValueError("Lists must have equal non-zero length")
    waits = [max(0, start - arrival) for arrival, start in zip(arrivals, starts)]
    return sum(waits) / len(waits)

print(average_wait_time([2, 5, 9], [4, 8, 10]))
```

## ۹.۶ تمرین ۴۹: یافتن طولانی‌ترین روند افزایشی پیوسته

طول بلندترین بخش پیوسته‌ای را بیابید که هر مقدار از مقدار قبلی بزرگ‌تر است.

```
def longest_increasing_run(numbers):
    if not numbers:
        return 0
    best = current = 1
    for index in range(1, len(numbers)):
        current = current + 1 if numbers[index] > numbers[index - 1] else 1
        best = max(best, current)
    return best

print(longest_increasing_run([2, 4, 7, 3, 5, 9, 12]))
```

## ۱۰.۶ تمرین ۵۰: تخصیص بودجه تناسبی

بودجه را متناسب با وزن واحدها توزیع کنید.

```
def allocate_budget(total, weights):
    weight_sum = sum(weights.values())
    if total < 0 or weight_sum <= 0:
        raise ValueError("Invalid allocation data")
```

```
    return {name: round(total * weight / weight_sum, 2)
            for name, weight in weights.items()}

print(allocate_budget(10000, {"Marketing": 3, "Development": 5, "Support": 2}))
```

# ۷ بخش ششم: اعتبارسنجی و مدیریت خطا

این تمرین‌ها ورودی نامعتبر را به‌صورت کنترل‌شده مدیریت می‌کنند.

## ۱.۷ تمرین ۵۱: دریافت عدد صحیح در بازه

تا دریافت عددی بین ۱ و ۱۰۰، ورودی را تکرار کنید.

```
while True:
    try:
        value = int(input("Enter an integer from 1 to 100: "))
        if 1 <= value <= 100:
            break
        print("Value is out of range")
    except ValueError:
        print("Enter a valid integer")
print(f"Accepted: {value}")
```

## ۲.۷ تمرین ۵۲: اعتبارسنجی شماره تلفن

پس از حذف فاصله و خط تیره، شماره باید دقیقاً ۱۰ رقم داشته باشد.

```
phone = input("Enter phone number: ").replace(" ", "").replace("-", "")
print("Valid phone number" if len(phone) == 10 and phone.isdigit() else "Invalid phone number")
```

## ۳.۷ تمرین ۵۳: اعتبارسنجی تاریخ میلادی

با `datetime` معتبر بودن تاریخ را بررسی کنید.

```
from datetime import datetime

date_text = input("Enter date (YYYY-MM-DD): ")
try:
    datetime.strptime(date_text, "%Y-%m-%d")
    print("Valid date")
except ValueError:
    print("Invalid date")
```

## ۴.۷ تمرین ۵۴: تقسیم امن داده‌های متنی

دو مقدار متنی را به عدد تبدیل و تقسیم کنید و خطاها را جداگانه مدیریت کنید.

```
try:
    numerator = float(input("Enter numerator: "))
    denominator = float(input("Enter denominator: "))
    print(f"Result: {numerator / denominator}")
except ValueError:
    print("Inputs must be numeric")
except ZeroDivisionError:
    print("Denominator cannot be zero")
```

## ۵.۷ تمرین ۵۵: اعتبارسنجی کد محصول

کد باید از سه حرف بزرگ، خط تیره و چهار رقم تشکیل شود.

```
import re

code = input("Enter product code: ")
valid = re.fullmatch(r"[A-Z]{3}-\d{4}", code) is not None
print("Valid product code" if valid else "Invalid product code")
```

## ۶.۷ تمرین ۵۶: دریافت انتخاب از منو

فقط گزینه‌های `add`، `edit` یا `delete` را بپذیرید.

```
allowed = {"add", "edit", "delete"}
while True:
    choice = input("Choose add, edit, or delete: ").strip().lower()
    if choice in allowed:
        print(f"Selected: {choice}")
        break
    print("Invalid choice")
```

## ۷.۷ تمرین ۵۷: اعتبارسنجی نشانی IPv4

چهار بخش عددی بین ۰ تا ۲۵۵ را بررسی کنید.

```
address = input("Enter IPv4 address: ")
parts = address.split(".")
valid = len(parts) == 4 and all(part.isdigit() and 0 <= int(part) <= 255 for part in parts)
print("Valid IPv4 address" if valid else "Invalid IPv4 address")
```

## ۸.۷ تمرین ۵۸: تبدیل مبلغ با پیام خطای دقیق

علامت پول و جداکننده هزارگان را حذف و مقدار را تبدیل کنید.

```
raw_amount = input("Enter amount: ").strip().replace(",", "").replace("$", "")
try:
    amount = float(raw_amount)
    if amount < 0:
        raise ValueError("Amount cannot be negative")
    print(f"Accepted amount: {amount:.2f}")
except ValueError as error:
    print(f"Invalid amount: {error}")
```

## ۹.۷ تمرین ۵۹: بررسی مسیر فایل ورودی

وجود فایل و فایل‌بودن مسیر را پیش از پردازش بررسی کنید.

```
from pathlib import Path

path = Path(input("Enter file path: "))
if not path.exists():
    print("Path does not exist")
elif not path.is_file():
    print("Path is not a file")
else:
    print(f"File size: {path.stat().st_size} bytes")
```

### ۱۰.۷ تمرین ۶۰: اعتبارسنجی رکورد کاربر

وجود فیلدهای ضروری و نوع سن را بررسی کنید.

```
user = {"name": "Alice", "email": "alice@example.com", "age": 28}
required = {"name", "email", "age"}
errors = []
if not required.issubset(user):
    errors.append("Missing required fields")
if "age" in user and (not isinstance(user["age"], int) or user["age"] < 0):
    errors.append("Invalid age")
print(errors if errors else "Valid user record")
```

## ۸ بخش هفتم: فایل، CSV و JSON

تمرین‌های این بخش داده را در قالب‌های رایج ذخیره و پردازش می‌کنند.

### ۱.۸ تمرین ۶۱: شمارش خطوط غیرخالی فایل

فایل نمونه بسازید و تعداد خطوط دارای محتوا را بشمارید.

```
from pathlib import Path

path = Path("notes.txt")
path.write_text("First note\n\nSecond note\nThird note\n", encoding="utf-8")
count = sum(1 for line in path.read_text(encoding="utf-8").splitlines() if line.strip())
print(f"Non-empty lines: {count}")
```

### ۲.۸ تمرین ۶۲: افزودن گزارش رویداد

رویداد جدید را همراه با زمان به انتهای فایل اضافه کنید.

```
from datetime import datetime

event = input("Enter event: ")
with open("events.log", "a", encoding="utf-8") as file:
    timestamp = datetime.now().isoformat(timespec="seconds")
    file.write(f"{timestamp} | {event}\n")
print("Event saved")
```

### ۳.۸ تمرین ۶۳: محاسبه مجموع فروش از CSV

یک CSV بسازید و مجموع ستون مبلغ را محاسبه کنید.

```
import csv

rows = [["product", "amount"], ["Book", "25.5"], ["Pen", "8.0"], ["Bag", "42.0"]]
with open("sales.csv", "w", newline="", encoding="utf-8") as file:
    csv.writer(file).writerows(rows)
with open("sales.csv", newline="", encoding="utf-8") as file:
    total = sum(float(row["amount"]) for row in csv.DictReader(file))
print(f"Total sales: {total:.2f}")
```

### ۴.۸ تمرین ۶۴: فیلتر CSV و ساخت فایل جدید

محصولات با موجودی کمتر از ۵ را در فایل جداگانه بنویسید.

```
import csv

products = [{"name": "Mouse", "stock": 3}, {"name": "Cable", "stock": 12}, {"name": "Monitor", "stock": 2}]
low_stock = [row for row in products if row["stock"] < 5]
with open("low_stock.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "stock"])
    writer.writeheader()
    writer.writerows(low_stock)
print(f"Rows written: {len(low_stock)}")
```

## ۵.۸ تمرین ۶۵: به‌روزرسانی تنظیمات JSON

تنظیمات موجود را بخوانید، یک مقدار را تغییر دهید و دوباره ذخیره کنید.

```
import json

settings = {"theme": "light", "notifications": True, "language": "en"}
with open("settings.json", "w", encoding="utf-8") as file:
    json.dump(settings, file, indent=2)
with open("settings.json", encoding="utf-8") as file:
    settings = json.load(file)
settings["theme"] = "dark"
with open("settings.json", "w", encoding="utf-8") as file:
    json.dump(settings, file, indent=2)
print("Settings updated")
```

## ۶.۸ تمرین ۶۶: پشتیبان‌گیری از فایل

از فایل متنی با پسوند `bak`. نسخه پشتیبان بسازید.

```
from pathlib import Path
import shutil

source = Path("report.txt")
source.write_text("Monthly report", encoding="utf-8")
backup = source.with_suffix(source.suffix + ".bak")
shutil.copy2(source, backup)
print(f"Backup created: {backup.name}")
```

## ۷.۸ تمرین ۶۷: یافتن بزرگ‌ترین فایل پوشه

در پوشه جاری، بزرگ‌ترین فایل را پیدا کنید.

```
from pathlib import Path

files = [path for path in Path(".").iterdir() if path.is_file()]
if files:
    largest = max(files, key=lambda path: path.stat().st_size)
    print(f"Largest file: {largest.name} ({largest.stat().st_size} bytes)")
else:
    print("No files found")
```

## ۸.۸ تمرین ۶۸: تبدیل CSV به JSON

رکوردهای CSV را به آرایه‌ای از شیءهای JSON تبدیل کنید.

```
import csv
import json

with open("sales.csv", newline="", encoding="utf-8") as source:
```

```
    records = list(csv.DictReader(source))
with open("sales.json", "w", encoding="utf-8") as target:
    json.dump(records, target, indent=2)
print(f"Converted records: {len(records)}")
```

### ۹.۸ تمرین ۶۹: استخراج کلمات خطا از گزارش

خطوط دارای ERROR را از فایل گزارش جدا کنید.

```
from pathlib import Path

log = Path("app.log")
log.write_text("INFO Started\nERROR Connection failed\nINFO Retrying\nERROR Timeout\n", encoding="utf-8")
errors = [line for line in log.read_text(encoding="utf-8").splitlines() if "ERROR" in line]
Path("errors.log").write_text("\n".join(errors), encoding="utf-8")
print(f"Errors extracted: {len(errors)}")
```

### ۱۰.۸ تمرین ۷۰: گزارش فراوانی پسوند فایل‌ها

تعداد فایل‌های هر پسوند را در پوشه جاری گزارش کنید.

```
from collections import Counter
from pathlib import Path

extensions = [path.suffix.lower() or "[no extension]" for path in Path(".").iterdir() if path.is_file()]
for extension, count in Counter(extensions).most_common():
    print(f"{extension}: {count}")
```

## ۹ بخش هشتم: شیءگرایی کاربردی

در این بخش، رفتار و داده مرتبط در قالب کلاس‌های کوچک مدل‌سازی می‌شوند.

### ۱.۹ تمرین ۷۱: کیف پول دیجیتال

کلاسی برای واریز، برداشت و نمایش موجودی بسازید.

```
class Wallet:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            return True
        return False

wallet = Wallet(100)
wallet.deposit(40)
print(wallet.withdraw(70), wallet.balance)
```

### ۲.۹ تمرین ۷۲: محصول با تخفیف

قیمت نهایی محصول را پس از اعمال تخفیف محاسبه کنید.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def discounted_price(self, percent):
        return round(self.price * (1 - percent / 100), 2)

product = Product("Headphones", 120)
print(product.discounted_price(15))
```

## ۳.۹ تمرین ۷۳: حساب بانکی با تاریخچه

تمام واریزها و برداشت‌های موفق را ثبت کنید.

```
class BankAccount:
    def __init__(self):
        self.balance = 0
        self.history = []

    def deposit(self, amount):
        self.balance += amount
        self.history.append(("deposit", amount))

    def withdraw(self, amount):
        if amount > self.balance:
            return False
        self.balance -= amount
        self.history.append(("withdraw", amount))
        return True

account = BankAccount()
account.deposit(500)
account.withdraw(125)
print(account.balance, account.history)
```

## ۴.۹ تمرین ۷۴: کتاب امانتی

کلاسی بسازید که وضعیت امانت کتاب را کنترل کند و از امانت دوباره جلوگیری کند.

```
class Book:
    def __init__(self, title):
        self.title = title
        self.borrowed = False

    def borrow(self):
        if self.borrowed:
            return False
        self.borrowed = True
        return True

    def return_book(self):
        self.borrowed = False

book = Book("Clean Code")
print(book.borrow())
print(book.borrow())
```

## ۵.۹ تمرین ۷۵: سبد خرید

محصولات را اضافه و مبلغ کل را محاسبه کنید.

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, name, price, quantity=1):
        self.items.append({"name": name, "price": price, "quantity": quantity})

    def total(self):
        return sum(item["price"] * item["quantity"] for item in self.items)

cart = ShoppingCart()
cart.add("Keyboard", 75, 2)
cart.add("Mouse", 30)
print(cart.total())
```

### ۶.۹ تمرین ۷۶: کارمند و حقوق خالص

حقوق خالص را پس از کسر درصد مالیات برگردانید.

```
class Employee:
    def __init__(self, name, gross_salary):
        self.name = name
        self.gross_salary = gross_salary

    def net_salary(self, tax_percent):
        return round(self.gross_salary * (1 - tax_percent / 100), 2)

employee = Employee("Alice", 4200)
print(employee.net_salary(12))
```

### ۷.۹ تمرین ۷۷: رزرو صندلی

کلاسی بسازید که از رزرو دوباره یک صندلی جلوگیری کند.

```
class SeatManager:
    def __init__(self, seat_count):
        self.available = set(range(1, seat_count + 1))

    def reserve(self, seat_number):
        if seat_number not in self.available:
            return False
        self.available.remove(seat_number)
        return True

manager = SeatManager(5)
print(manager.reserve(3))
print(manager.reserve(3))
```

### ۸.۹ تمرین ۷۸: دماسنج با ویژگی محاسباتی

دمای سلسیوس را ذخیره و فارنهایت را با `property` ارائه کنید.

```
class Thermometer:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

thermometer = Thermometer(25)
print(thermometer.fahrenheit)
```

### ۹.۹ تمرین ۷۹: اعلان‌های چندنوعی

با چندریختی، پیام را در کانال‌های مختلف ارسال کنید.

```
class EmailNotifier:
    def send(self, message):
        print(f"Email sent: {message}")

class SmsNotifier:
    def send(self, message):
        print(f"SMS sent: {message}")

for notifier in [EmailNotifier(), SmsNotifier()]:
    notifier.send("Your order is ready")
```

### ۱۰.۹ تمرین ۸۰: دانشجو با میانگین وزنی

نمره‌ها و وزن‌ها را نگه دارید و میانگین وزنی را محاسبه کنید.

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, score, weight):
        self.grades.append((score, weight))

    def weighted_average(self):
        total_weight = sum(weight for _, weight in self.grades)
        return sum(score * weight for score, weight in self.grades) / total_weight

student = Student("Bob")
student.add_grade(85, 2)
student.add_grade(92, 3)
print(student.weighted_average())
```

## ۱۰ بخش نهم: ساختمان داده و حل مسئله

این بخش از ساختمان‌داده‌های مناسب برای حل کارآمد مسئله‌ها استفاده می‌کند.

### ۱.۱۰ تمرین ۸۱: صف پشتیبانی

درخواست‌ها را به ترتیب ورود پردازش کنید.

```
from collections import deque

tickets = deque(["Login issue", "Payment issue", "Password reset"])
while tickets:
    ticket = tickets.popleft()
    print(f"Processing: {ticket}")
```

### ۲.۱۰ تمرین ۸۲: تاریخچه مرورگر با پشته

عملیات بازگشت به صفحه قبلی را شبیه‌سازی کنید.

```
history = []
current = "home"
for page in ["products", "cart", "checkout"]:
    history.append(current)
```

```
    current = page
print(f"Current: {current}")
if history:
    current = history.pop()
print(f"After back: {current}")
```

### ۳.۱۰ تمرین ۸۳: جست‌وجوی دودویی

اندیس هدف را در فهرست مرتب پیدا کنید.

```
def binary_search(numbers, target):
    left, right = 0, len(numbers) - 1
    while left <= right:
        middle = (left + right) // 2
        if numbers[middle] == target:
            return middle
        if numbers[middle] < target:
            left = middle + 1
        else:
            right = middle - 1
    return -1

print(binary_search([3, 8, 12, 19, 27, 31], 19))
```

### ۴.۱۰ تمرین ۸۴: مرتب‌سازی انتخابی

مرتب‌سازی صعودی را بدون استفاده از `sort` پیاده‌سازی کنید.

```
def selection_sort(numbers):
    result = numbers[:]
    for index in range(len(result)):
        smallest = index
        for candidate in range(index + 1, len(result)):
            if result[candidate] < result[smallest]:
                smallest = candidate
        result[index], result[smallest] = result[smallest], result[index]
    return result

print(selection_sort([9, 2, 7, 4, 1]))
```

### ۵.۱۰ تمرین ۸۵: پیمایش شبکه با BFS

کمترین تعداد حرکت از مبدأ تا مقصد deque در یک شبکه بدون مانع محاسبه کنید.

```
from collections import deque

def shortest_path(rows, cols, start, target):
    queue = deque([(start, 0)])
    visited = {start}
    while queue:
        (row, col), distance = queue.popleft()
        if (row, col) == target:
            return distance
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            next_cell = (row + dr, col + dc)
            if 0 <= next_cell[0] < rows and 0 <= next_cell[1] < cols and next_cell not in visited:
                visited.add(next_cell)
                queue.append((next_cell, distance + 1))
    return -1

print(shortest_path(4, 5, (0, 0), (3, 4)))
```

### ۶.۱۰ تمرین ۸۶: شمارش فراوانی با Counter

سه واژه پرتکرار یک متن را نمایش دهید.

```
from collections import Counter

words = input("Enter text: ").lower().split()
for word, count in Counter(words).most_common(3):
    print(f"{word}: {count}")
```

### ۷.۱۰ تمرین ۸۷: صف اولویت کارها

کارها را بر اساس اولویت کمتر پردازش کنید.

```
import heapq

tasks = []
heapq.heappush(tasks, (2, "Write report"))
heapq.heappush(tasks, (1, "Fix production bug"))
heapq.heappush(tasks, (3, "Update documentation"))
while tasks:
    priority, task = heapq.heappop(tasks)
    print(f"Priority {priority}: {task}")
```

### ۸.۱۰ تمرین ۸۸: حافظه نهان ساده

نتیجه محاسبات تکراری را در دیکشنری ذخیره کنید.

```
cache = {}

def shipping_cost(weight):
    if weight not in cache:
        cache[weight] = 5 + weight * 1.75
    return cache[weight]

print(shipping_cost(4))
print(shipping_cost(4))
print(cache)
```

### ۹.۱۰ تمرین ۸۹: یافتن مسیر در گراف

با DFS مشخص کنید آیا میان دو گره مسیر وجود دارد.

```
def has_path(graph, start, target):
    stack = [start]
    visited = set()
    while stack:
        node = stack.pop()
        if node == target:
            return True
        if node not in visited:
            visited.add(node)
            stack.extend(graph.get(node, []))
    return False

graph = {"A": ["B", "C"], "B": ["D"], "C": [], "D": ["E"], "E": []}
print(has_path(graph, "A", "E"))
```

### ۱۰.۱۰ تمرین ۹۰: بیشترین درآمد بدون انتخاب روزهای مجاور

بیشترین مجموع را با این شرط بیابید که دو عنصر مجاور انتخاب نشوند.

```
def max_non_adjacent_sum(values):
    include = exclude = 0
    for value in values:
        include, exclude = exclude + value, max(include, exclude)
    return max(include, exclude)

print(max_non_adjacent_sum([6, 10, 12, 7, 9, 14]))
```

# ۱۱ بخش دهم: مینی‌پروژه‌های کاربردی

هر پروژه زیر چند مفهوم را در یک برنامه کوتاه و قابل‌اجرا ترکیب می‌کند.

## ۱.۱۱ تمرین ۹۱: مدیریت هزینه‌های روزانه

چند هزینه را دریافت، مجموع و گران‌ترین مورد را گزارش کنید.

```
expenses = []
count = int(input("Enter expense count: "))
for _ in range(count):
    title = input("Enter expense title: ")
    amount = float(input("Enter expense amount: "))
    expenses.append({"title": title, "amount": amount})
if expenses:
    total = sum(item["amount"] for item in expenses)
    highest = max(expenses, key=lambda item: item["amount"])
    print(f"Total: {total:.2f}")
    print(f"Highest: {highest['title']} - {highest['amount']:.2f}")
else:
    print("No expenses recorded")
```

## ۲.۱۱ تمرین ۹۲: فهرست کارها

کارها را اضافه، تکمیل و وضعیت نهایی را نمایش دهید.

```
tasks = []
while True:
    command = input("Enter add, done, list, or quit: ").lower()
    if command == "add":
        tasks.append({"title": input("Enter task: "), "done": False})
    elif command == "done":
        index = int(input("Enter task number: ")) - 1
        if 0 <= index < len(tasks):
            tasks[index]["done"] = True
    elif command == "list":
        for index, task in enumerate(tasks, 1):
            status = "Done" if task["done"] else "Pending"
            print(f"{index}. [{status}] {task['title']}")
    elif command == "quit":
        break
```

## ۳.۱۱ تمرین ۹۳: نظرسنجی ساده

رأی‌ها را دریافت و برنده را مشخص کنید.

```
from collections import Counter

votes = []
while True:
    vote = input("Enter option or finish: ").strip()
```

```
    if vote.lower() == "finish":
        break
    if vote:
        votes.append(vote)
counts = Counter(votes)
if counts:
    winner, total = counts.most_common(1)[0]
    print(f"Winner: {winner} with {total} votes")
else:
    print("No votes recorded")
```

## ۴.۱۱ تمرین ۹۴: دفترچه مخاطبان

افزودن و جست‌وجوی مخاطب را پیاده‌سازی کنید.

```
contacts = {}
while True:
    command = input("Enter add, find, list, or quit: ").lower()
    if command == "add":
        name = input("Enter name: ").strip()
        contacts[name.lower()] = input("Enter phone: ").strip()
    elif command == "find":
        name = input("Enter name: ").strip().lower()
        print(contacts.get(name, "Contact not found"))
    elif command == "list":
        for name, phone in sorted(contacts.items()):
            print(f"{name.title()}: {phone}")
    elif command == "quit":
        break
```

## ۵.۱۱ تمرین ۹۵: سیستم ورود با قفل موقت

سه تلاش برای ورود در نظر بگیرید و ورود موفق را ثبت کنید.

```
users = {"alice": "blue42", "bob": "code99"}
username = input("Enter username: ").lower()
if username not in users:
    print("Unknown user")
else:
    for attempt in range(3):
        password = input("Enter password: ")
        if password == users[username]:
            print("Login successful")
            break
        print(f"Attempts left: {2 - attempt}")
    else:
        print("Account temporarily locked")
```

## ۶.۱۱ تمرین ۹۶: صدور فاکتور فروش

اقلام را دریافت و جمع جزء، مالیات و مبلغ نهایی را چاپ کنید.

```
items = []
count = int(input("Enter item count: "))
for _ in range(count):
    name = input("Enter item name: ")
    price = float(input("Enter unit price: "))
    quantity = int(input("Enter quantity: "))
    items.append((name, price, quantity))
subtotal = sum(price * quantity for _, price, quantity in items)
tax = subtotal * 0.09
```

```
for name, price, quantity in items:
    print(f"{name}: {quantity} x {price:.2f} = {price * quantity:.2f}")
print(f"Subtotal: {subtotal:.2f}")
print(f"Tax: {tax:.2f}")
print(f"Total: {subtotal + tax:.2f}")
```

### ۷.۱۱ تمرین ۹۷: تحلیل نمرات کلاس

میانگین کلاس، تعداد قبولی‌ها و بهترین دانش‌آموز را گزارش کنید.

```
students = {}
count = int(input("Enter student count: "))
for _ in range(count):
    name = input("Enter student name: ")
    students[name] = float(input("Enter score: "))
if students:
    average = sum(students.values()) / len(students)
    passed = sum(score >= 60 for score in students.values())
    best = max(students, key=students.get)
    print(f"Class average: {average:.2f}")
    print(f"Passed students: {passed}")
    print(f"Top student: {best} - {students[best]:.2f}")
```

### ۸.۱۱ تمرین ۹۸: رزرو اتاق هتل

اتاق آزاد را رزرو کنید و از رزرو اتاق اشغال جلوگیری کنید.

```
rooms = {101: "available", 102: "occupied", 103: "available"}
while True:
    command = input("Enter reserve, status, or quit: ").lower()
    if command == "reserve":
        room = int(input("Enter room number: "))
        if rooms.get(room) == "available":
            rooms[room] = "occupied"
            print("Reservation confirmed")
        else:
            print("Room is unavailable")
    elif command == "status":
        for room, status in sorted(rooms.items()):
            print(f"Room {room}: {status}")
    elif command == "quit":
        break
```

### ۹.۱۱ تمرین ۹۹: تحلیل کپشن

تعداد کلمات، اشاره‌ها، هشتگ‌ها و طولانی‌ترین کلمه را گزارش کنید.

```
import re

caption = input("Enter caption: ")
words = re.findall(r"[A-Za-z0-9_']+", caption)
hashtags = re.findall(r"#[A-Za-z0-9_]+", caption)
mentions = re.findall(r"@[A-Za-z0-9_]+", caption)
longest = max(words, key=len) if words else ""
print(f"Words: {len(words)}")
print(f"Hashtags: {len(hashtags)}")
print(f"Mentions: {len(mentions)}")
print(f"Longest word: {longest}")
```

## ۱۰.۱۱ تمرین ۱۰۰: مدیریت سفارش رستوران

منو، سفارش چند قلم و مبلغ نهایی را مدیریت کنید.

```
menu = {"burger": 8.5, "pizza": 12.0, "salad": 6.5, "juice": 3.0}
order = {}
while True:
    item = input("Enter menu item or finish: ").lower()
    if item == "finish":
        break
    if item not in menu:
        print("Item not found")
        continue
    quantity = int(input("Enter quantity: "))
    if quantity > 0:
        order[item] = order.get(item, 0) + quantity
subtotal = sum(menu[item] * quantity for item, quantity in order.items())
service = subtotal * 0.10
for item, quantity in order.items():
    print(f"{item.title()}: {quantity} x {menu[item]:.2f}")
print(f"Subtotal: {subtotal:.2f}")
print(f"Service charge: {service:.2f}")
print(f"Total: {subtotal + service:.2f}")
```